

Informe técnico: sistema de monitoreo IoT con análisis predictivo para ganadería

1. Descripción del caso de uso

1.1 Contexto y problemática

En operaciones ganaderas modernas, la salud y el bienestar animal son críticos para la rentabilidad. Tradicionalmente, el monitoreo del consumo de alimento se realiza de manera manual: observación visual, pesajes puntuales, registros en papel. Este enfoque presenta limitaciones:

- **Falta de detección temprana:** cambios sutiles en consumo pueden indicar enfermedad, pero pasan desapercibidos
- **Ineficiencia operativa:** peones deben realizar inspecciones periódicas manuales
- **Imposibilidad de análisis:** sin datos históricos granulares, no se pueden identificar tendencias
- **Imposibilidad de comparación:** cada animal en su contexto, sin benchmark

1.2 Solución propuesta

Se diseña un **sistema de datos** que sustenta una aplicación IoT para monitoreo continuo de consumo individual mediante:

1. **Comederos y bebederos inteligentes** equipados con sensores RFID y pesaje
2. **Captura automática** de mediciones: qué animal comió, cuánto, cuándo
3. **Almacenamiento centralizado** de millones de registros históricos
4. **Detección de anomalías** mediante análisis de patrones de consumo
5. **Generación de alertas** automáticas ante cambios relevantes

1.3 Actores del sistema

- **Peones:** operarios que revisan corrales, resuelven alertas
- **Encargado de producción:** supervisa múltiples ubicaciones, analiza tendencias
- **Veterinario:** accede a datos para diagnóstico y tratamiento
- **Administrador:** gestiona dispositivos, usuarios, configuraciones

1.4 Objetivos de la solución

Objetivos técnicos de datos:

1. Almacenar mediciones de forma eficiente (10-100 registros/segundo por estancia)
2. Consultar históricos completos con latencia aceptable (<1 segundo)
3. Detectar eventos anómalos en tiempo real
4. Generar alertas automáticas según reglas configurables
5. Escalar sin degradación de performance ante crecimiento de datos

Objetivos empresariales:

1. Anticipar problemas de salud animal

2. Optimizar consumo de alimento
 3. Reducir mortalidad y morbilidad
 4. Mejorar eficiencia operativa
 5. Facilitar toma de decisiones basada en datos
-

2. Relevamiento de datos necesarios

2.1 Entidades principales del dominio

USUARIOS

Personas que interactúan con el sistema.

- Identificador único
- Nombre, email, teléfono
- Rol: determina permisos (peón, encargado, veterinario, admin)
- Estancia asociada: cada usuario pertenece a una estancia (multi-tenancy)
- Fecha de activación/desactivación

ESTANCIAS

Unidades ganaderas (empresas/predios).

- Identificador único
- Nombre, descripción
- Ubicación geográfica
- Tipo de producción: engorde, cría, lechería
- Cantidad de animales actual
- Contacto responsable

UBICACIONES

Sectores físicos donde se instalan dispositivos (corrales, encierre, galpones).

- Identificador único
- Estancia a la que pertenece
- Nombre descriptivo
- Tipo: corral, encierre, galpón
- Capacidad máxima de animales
- Responsable designado

DISPOSITIVOS

Comederos y bebederos inteligentes instalados en ubicaciones.

- Identificador único (serial del dispositivo)
- Ubicación donde está instalado
- Tipo: comedero, bebedero
- Modelo y fabricante

- Fecha de instalación/retiro
- Estado: operacional, en mantenimiento, falla
- Configuración en JSON: umbrales, parámetros de calibración

SENSORES

Componentes de medición dentro de los dispositivos.

- Identificador único
- Dispositivo al que pertenece
- Tipo: RFID, pesaje, temperatura, humedad
- Unidad de medida
- Rango operativo
- Precisión declarada
- Estado: operacional, descalibrado, falla

ANIMALES

Entidades ganaderas individuales.

- Identificador único (ID interno)
- Tag RFID único: identificador que lee el sensor
- Estancia a la que pertenece
- Nombre o alias
- Raza, sexo
- Fecha de nacimiento / ingreso a estancia
- Peso actual
- Estado de salud: sano, enfermo, en tratamiento, descarte
- Fecha de egreso (NULL si sigue en predios)

MEDICIONES

Registros de eventos de consumo. **Esta es la tabla más crítica y grande.**

- Identificador único
- Timestamp exacto del evento
- Dispositivo que registró
- Sensor que capturó
- Animal identificado (puede ser NULL si falla RFID)
- Valor medido: cantidad consumida en kg
- Duración del evento en segundos
- Temperatura y humedad ambiental (opcional)
- Flag: **es_anomalia** (calculado posteriormente)

ALERTAS

Eventos detectados por lógica de reglas.

- Identificador único

- Animal asociado (o dispositivo si es alert de hardware)
- Tipo de alerta: bajo_consumo, no_presencia, falla_sensor, consumo_atípico
- Severidad: baja, media, alta, crítica
- Timestamp de generación
- Descripción explicativa
- Estado: abierta, en_progreso, resuelta
- Usuario responsable (quién la resuelve)
- Timestamp de resolución (NULL si no resuelta)

2.2 Relaciones principales

Origen	Destino	Tipo	Cardinalidad	Justificación
USUARIOS	ESTANCIAS	FK	N:1	Un usuario pertenece a una estancia (multi-tenancy)
ESTANCIAS	UBICACIONES	FK	1:N	Una estancia tiene múltiples corrales/sectores
UBICACIONES	DISPOSITIVOS	FK	1:N	Una ubicación tiene comederos/bebederos
DISPOSITIVOS	SENSORES	FK	1:N	Un dispositivo tiene múltiples sensores
SENSORES	MEDICIONES	FK	1:N	Un sensor genera millones de mediciones
DISPOSITIVOS	MEDICIONES	FK	1:N	Un dispositivo registra todas sus mediciones
ANIMALES	MEDICIONES	FK	N:1	Un animal tiene múltiples registros de consumo
ANIMALES	ALERTAS	FK	1:N	Un animal puede generar múltiples alertas
DISPOSITIVOS	ALERTAS	FK	0:N	Un dispositivo puede generar alertas (falla)
USUARIOS	ALERTAS	FK	0:N	Un usuario resuelve alertas

2.3 Restricciones de integridad

Restricciones de unicidad:

- **tag_id** en ANIMALES: identificador RFID único en estancia
- **serial** en DISPOSITIVOS: identificador de hardware único globalmente
- Email en USUARIOS: email único por estancia

Restricciones de formato:

- Tag RFID: formato hexadecimal, longitud fija
- Email: formato válido
- Temperaturas: rango -40°C a +50°C
- Consumo: valores positivos

Restricciones temporales:

- Mediciones deben estar ordenadas temporalmente

- Alertas resueltas tienen timestamp de resolución > timestamp de alerta

2.4 Flujos de datos principales

Flujo 1: Captura de medición

```
Dispositivo RFID → Lee tag animal
      ↓
Sensor pesaje → Registra peso consumido
      ↓
Base de datos MEDICIONES ← timestamp, animal_id, cantidad
      ↓
Trigger calcula promedio → Evalúa anomalía
      ↓
Si anomalía → Crea registro en ALERTAS
```

Flujo 2: Resolución de alerta

```
Peón ve alerta → Inspecciona animal
      ↓
Diagnóstico (sano/enfermo)
      ↓
Peón resuelve alerta en sistema
      ↓
Sistema registra resolución: timestamp, usuario_id
```

3. Clasificación de los datos según su tipo

3.1 Datos estructurados (80% del volumen)

Son datos que responden a un esquema riguroso, almacenables en tablas relacionales.

Entidades core:

- USUARIOS, ESTANCIAS, UBICACIONES
- DISPOSITIVOS, SENSORES
- ANIMALES
- MEDICIONES (la mayor parte: timestamp, valor numérico, IDs)
- ALERTAS

Características:

- Esquema fijo predefinido
- Relaciones claras
- Queryables mediante SQL standard
- Altos requerimientos de integridad referencial
- Transacciones ACID

Volumen estimado:

- MEDICIONES: 10-100 registros/segundo por estancia
- En 1 año: 315M a 3.15B registros
- Tamaño: 30-300 GB/año por estancia

3.2 Datos semi-estructurados (15% del volumen)

Datos que varían en estructura, almacenados en JSONB.

Ejemplos:

- **DISPOSITIVOS.configuracion**: parámetros específicos del modelo

```
{
  "calibracion_pesaje": 0.98,
  "umbral_presencia": 100,
  "intervalo_reporte": 60
}
```

- **MEDICIONES.datos_crudos**: información adicional del evento

```
{
  "rssi_rfid": -65,
  "calidad_lectura": 95,
  "numero_intentos_lectura": 2
}
```

Características:

- Estructura variable según contexto
- Queryable parcialmente en SQL (operadores JSONB)
- Permite evolución sin migración

Volumen estimado:

- ~200 bytes por medición: 60-600 GB/año

3.3 Datos no estructurados (5% del volumen)

Datos libres de formato, generalmente texto.

Ejemplos:

- Notas de resolución de alertas: texto libre
- Observaciones de veterinario: notas médicas

Características:

- Sin esquema predefinido

- Almacenables en columnas TEXT
- Indexables mediante full-text search

Volumen estimado:

- ~100 bytes por alerta de texto

3.4 Datos vectoriales (1% del volumen, pero crítico para IA)

Representaciones numéricas multidimensionales para búsqueda por similitud.

Ejemplos:

- **Embedding de patrón de consumo:** vector de 768D que representa "firma" de cómo come un animal
 - Entrenado sobre secuencias de 30 mediciones consecutivas
 - Permite encontrar animales "similares" en comportamiento

Casos de uso:

1. **Detección de anomalías:** comparar embedding actual contra histórico
2. **Detección de epidemias:** encontrar múltiples animales con patrones anómalos similares
3. **Predicción de problemas:** clusters de animales con riesgo similar

Características:

- Generados por modelos de embedding
- Almacenados en columnas vectoriales (pgvector)
- Queryables mediante búsqueda de similitud

Volumen estimado:

- 768 dimensiones × 4 bytes = ~3KB por vector
- Si se genera 1 vector por animal/día: 3KB × 10K animales × 365 días = ~11 GB/año

3.5 Matriz de almacenamiento recomendado

Tipo de dato	Dónde almacenar	Tecnología	Justificación
Estructurados	PostgreSQL (tablas normalizadas)	SQL relacional	ACID, integridad, queries complejas
Semi-estructurados	PostgreSQL (JSONB)	Column type JSONB	Flexibilidad sin migración
No estructurados (texto)	PostgreSQL (TEXT)	Text column	Indexable con full-text search
Vectoriales	PostgreSQL (pgvector)	Vector type + índices	Co-ubicado con datos relacionales

Conclusión de secciones 1-3

Se han identificado **8 entidades principales** con **10 relaciones clave**, almacenando datos de **4 tipos diferentes** (estructurados, semi, no estructurados, vectoriales). El volumen estimado es de **30-600 GB/año** en mediciones estructuradas.

4. Modelo conceptual

El modelo conceptual completo (diagrama entidad-relación en representación textual, atributos por entidad con tipo/restricciones/justificación, matriz de cardinalidades y restricciones de integridad) está en [docs/01_modelo_conceptual.md](#). En síntesis: 8 entidades — **estancias** (tenant raíz), **usuarios**, **ubicaciones**, **dispositivos**, **sensores**, **animales**, **mediciones** (tabla de hechos) y **alertas** — conectadas por relaciones 1:N que forman dos cadenas de dependencia: una organizacional (**estancias** → **ubicaciones** → **dispositivos** → **sensores**) y una operacional (**animales/dispositivos/sensores** → **mediciones** → **alertas**).

5. Modelo de implementación según tecnología elegida

Se eligió un modelo **relacional normalizado en PostgreSQL 17**, con la extensión **pgvector** para la porción vectorial (no una base vectorial separada — justificado en la sección 11 y en [vectorial/modelo_vectorial.md](#)). El modelo lógico completo, con cada tabla derivada del modelo conceptual y su verificación de forma normal, está en [docs/02_modelo_logico.md](#); el modelo físico (tipos concretos, particionamiento, índices, RLS, triggers) en [docs/03_modelo_fisico.md](#). No se usó ningún paradigma NoSQL adicional: el propio caso de uso (datos estructurados con relaciones estables e integridad referencial crítica — una medición sin animal o dispositivo válido no tiene sentido) encaja mejor en relacional que en documental/clave-valor/columnar/grafos, según se argumenta en la sección 11 de [docs/02_modelo_logico.md](#).

6. Decisiones de normalización, embebido, referencia o desnormalización

El esquema está en **3NF** con **tres desnormalizaciones controladas**, cada una justificada por una necesidad de consulta frecuente y con su mecanismo de consistencia (trigger) explícito (detalle completo, con el análisis de anomalías de inserción/actualización/eliminación que evitan, en [docs/02_modelo_logico.md](#), secciones 4 y 5):

1. **estancias.cantidad_animales_actual**: evita un **COUNT** contra **animales** en cada lectura del dashboard. Mantenido por el trigger **tg_actualizar_cantidad_animales**.
2. **animales.ubicacion_actual_id**: evita resolver la ubicación actual con un **JOIN** de 3 niveles contra **mediciones** (tabla de millones de filas). Mantenido por **tg_actualizar_ubicacion_animal**.
3. **Columnas JSONB** (**dispositivos.configuracion**, **mediciones.datos_crudos**, **alertas.datos_contextuales**): datos cuya estructura varía por modelo de hardware o por tipo de evento, y que no se filtran en las consultas representativas — normalizarlos en tablas separadas agregaría joins sin beneficio de consulta real.

No hay "embebido" en el sentido NoSQL porque no se usó un motor documental; el equivalente relacional de esa decisión es precisamente el uso de JSONB descrito arriba, en vez de romper esos atributos variables en tablas 1:1 adicionales.

7. Justificación de la tecnología seleccionada

PostgreSQL + pgvector se eligió sobre las alternativas evaluadas por estas razones concretas al caso de uso (comparación completa contra MongoDB en [docs/02_modelo_logico.md](#), sección 8):

- **Integridad referencial no negociable:** una medición sin `dispositivo_id/sensor_id` válido, o una alerta sin `animal_id` ni `dispositivo_id`, son estados inválidos del dominio — el motor debe rechazarlos, no la aplicación. Un motor documental no da esa garantía nativa.
- **Transacciones ACID entre tablas relacionadas:** el flujo medición → detección de anomalía → alerta (sección 8) debe ser atómico; si la detección de anomalía falla a mitad de camino, no puede quedar una medición "huérfana" sin su alerta correspondiente.
- **RLS nativo para multi-tenancy** (sección 13): resuelve el aislamiento entre estancias en el motor, no en cada punto de la aplicación que consulta datos — una sola superficie de fuga posible en vez de una por cada lugar del código que arma una query.
- **pgvector evita un segundo sistema:** el volumen de vectores del caso de uso (un patrón diario por animal, no una vectorización de cada evento) es bajo — no justifica operar una base vectorial dedicada además de la relacional (comparación de criterios en [vectorial/modelo_vectorial.md](#), sección 6).
- **Particionamiento y EXPLAIN predecible:** con `mediciones` proyectada a cientos de millones de filas, se necesita control fino sobre el plan de ejecución (partition pruning, índices parciales) — más difícil de razonar en un motor sin optimizador basado en costos maduro para consultas relacionales complejas.

El costo aceptado: PostgreSQL no escala horizontalmente "gratis" como un documental nativo — si el volumen supera lo que una instancia sostiene, la vía es sharding manual por `estancia_id` (ver [arquitectura/escalabilidad.md](#), sección 5), no un mecanismo automático del motor.

8. Implementación mínima realizada

La implementación es SQL real contra PostgreSQL 17 con `pgvector/pgvector:pg17`, levantado con `docker-compose up -d`:

Archivo	Contenido	Comportamiento esperado
db/estructura/schema.sql	8 tablas, constraints, <code>mediciones</code> declarada <code>PARTITION BY RANGE</code>	Crea las tablas sin errores de integridad
db/estructura/particiones.sql	12 particiones mensuales de 2026 + partición <code>DEFAULT</code>	Las 353 filas de ejemplo se distribuyen correctamente por mes
db/estructura/indexes.sql	Índices por FK, parciales (anomalías, alertas activas) y vectorial (<code>ivfflat</code>)	Se propagan a todas las particiones
db/estructura/triggers.sql	Desnormalización + detección de anomalías + alertas automáticas	Ver corrección de falso-positivo en sección 6 de docs/03_modelo_fisico.md
db/estructura/rls.sql	Rol <code>app_user</code> + políticas RLS por estancia (incluida <code>estancias</code>)	Aísla correctamente: estancia 2 solo accede a sus propios animales, nunca a los de estancia 1

Archivo	Contenido	Comportamiento esperado
<code>db/estructura/views.sql</code>	4 vistas para consultas frecuentes	Usadas por la consulta representativa #7
<code>db/vectorial/embeddings.sql</code>	Función <code>animales_similares()</code> sobre <code><=></code> (coseno)	Devuelve un ranking coherente (ver sección 10)
<code>data/ejemplos/datos_simulados.sql</code>	Carga completa (sección 9)	353 mediciones, 8 alertas, ~168 vectores

Orden de ejecución y por qué importa (particiones antes que índices, todo antes que RLS y datos):

`docs/03_modelo_fisico.md`, sección 11.

9. Datos de ejemplo utilizados

`data/ejemplos/datos_simulados.sql` genera, de forma reproducible (`setseed`) y relativa a la fecha de ejecución (no fechas fijas):

- **Catálogo:** 2 estancias (una de engorde, una lechera — para poder demostrar aislamiento multi-tenant con datos reales en ambas), 6 usuarios con los 4 roles del dominio, 5 ubicaciones, 6 dispositivos, 11 sensores, 8 animales.
- **353 mediciones:** 336 generadas por `generate_series` cubriendo 21 días previos (2 eventos/día por animal, con ruido $\pm 12\%$ sobre un consumo base propio de cada animal) + 17 mediciones "de hoy" escritas a mano como ejemplo legible. El animal 5 (`en_tratamiento`) tiene una caída de consumo progresiva y deliberada en los últimos 7 días (de ~12kg a ~5kg), para que el trigger de detección de anomalías tenga un caso real que detectar — no ruido aleatorio que podría o no cruzar el umbral.
- **~168 vectores de patrón de consumo** (uno por animal/día con datos), sintéticos: valores bajos y homogéneos para animales sanos, desplazados hacia arriba para el animal en tratamiento durante su semana de caída — alcanza para que una búsqueda de similitud agrupe correctamente sanos vs. enfermo (justificado en `vectorial/modelo_vectorial.md`, sección 1).
- **8 alertas:** 3 cargadas a mano + 5 generadas automáticamente por el trigger al insertar las mediciones del animal 5, mostrando el disparo del trigger sobre datos con la caída de consumo real, no solo sobre su definición.

El criterio de carga no busca casos que violen las restricciones del schema: las mediciones, alertas y el resto del catálogo se generan siempre dentro de los rangos válidos de cada CHECK, UNIQUE y FK, de modo que la carga completa respete la integridad definida en el modelo físico.

10. Consultas representativas

Las 8 consultas de `db/consultas/queries_representativas.sql` (todas probadas contra los datos de ejemplo, conectando como `app_user` salvo donde se indica lo contrario):

1. **Consumo total/promedio por animal (7 días)** — filtrado + agregación; vista base de un encargado.
2. **Animales por debajo de su propio histórico** — compara promedio de 24h vs. 7 días previos con CTEs; detección de salud sin umbral fijo global.
3. **Consumo promedio por ubicación** — JOIN de 3 niveles; distingue problema de comedero vs. problema de animal.

4. **Alertas sin resolver, priorizadas** — usa el índice parcial `idx_alertas_estado_timestamp`; bandeja de trabajo operativa.
5. **Similitud vectorial** — `<=>` (coseno) sobre `idx_mediciones_embedding`; encuentra animales con patrón parecido al de referencia.
6. **Promedio móvil de 5 eventos (función de ventana)** — `AVG() OVER (ROWS BETWEEN 4 PRECEDING AND CURRENT ROW)`; vista de detalle para veterinario.
7. **Alertas activas por estancia** (vista `alertas_activas_por_estancia`, consultada como `postgres`) — panorama multi-tenant, excepción intencional al aislamiento por RLS, pensada para administración de plataforma.
8. **Trazabilidad de un caso** — `UNION ALL` de mediciones anómalas y alertas de un animal en una sola línea de tiempo; auditoría.

Cada una responde una pregunta operativa real del caso de uso (no son variaciones triviales de `SELECT *`) y juntas cubren selección/filtrado, joins multi-tabla, agregación, funciones de ventana, y la consulta que justifica un índice — el mínimo pedido por la consigna.

11. Propuesta para datos semiestructurados, no estructurados y vectoriales

- **Semiestructurados** (JSONB): `dispositivos.configuracion`, `mediciones.datos_crudos`, `alertas.datos_contextuales` — justificado en la sección 6.
- **No estructurados**: notas de resolución de alerta (`alertas.notas_resolucion`, `TEXT` libre) y observaciones de veterinario — bajo volumen en este caso de uso, no se justificó un motor de búsqueda de texto completo dedicado; un índice GIN con `to_tsvector` alcanzaría si en el futuro se necesitara buscar por contenido de las notas.
- **Vectoriales**: desarrollado en profundidad en `vectorial/modelo_vectorial.md` — qué se vectoriza (patrón diario de consumo, no cada evento), dónde vive (columna de `mediciones`, no un almacén separado), qué consultas resuelve, y cómo hereda el aislamiento RLS.

12. Propuesta de arquitectura de datos

Desarrollada en `docs/04_arquitectura_datos.md`: arquitectura de una sola base operacional (no Data Lake/Warehouse/Lakehouse) con separación **lógica** por capas de procesamiento (cruda → procesada por triggers → preparada para IA), justificada por el bajo volumen y la necesidad de respuesta en segundos sobre datos recientes, no de análisis batch sobre años de historia. Incluye el diagrama de flujo desde el sensor hasta cada tipo de consumidor (app operativa filtrada por RLS, vistas ejecutivas multi-tenant, análisis puntual).

13. Estrategia de seguridad, permisos y aislamiento

Implementado: aislamiento multi-tenant por estancia vía RLS (`db/estructura/rls.sql`), con un rol de aplicación (`app_user`) sin privilegio de superusuario/bypass — condición necesaria para que RLS aplique (ver `docs/03_modelo_fisico.md`, sección 6, sobre por qué probar RLS conectado como `postgres` es engañoso). El diseño es fail-closed: sin `SET app.estancia_id`, las consultas fallan en vez de devolver datos de todas las estancias. Por ejemplo, un usuario con `app.estancia_id = 2` accede a 2 animales y 84 mediciones, mientras que con `app.estancia_id = 1` accede a 6 animales y 269 mediciones, sin que un contexto pueda ver los datos del otro.

Las políticas cubren las tres operaciones que la aplicación necesita sobre cada tabla (`SELECT`, `INSERT`, `UPDATE`) — no solo lectura: un usuario puede registrar un sensor nuevo, corregir una medición o resolver una

alerta, siempre y cuando esa fila resuelva, por la cadena de FK correspondiente, a su propia estancia. No hay política de **DELETE** porque el modelo no la necesita: los estados se dan de baja lógicamente (**activo**, **estado**, **fecha_egreso_***) en vez de borrarse, consistente con el resto del diseño (sección 6).

Limitación reconocida y no implementada en esta entrega: RLS aísla por *estancia*, no por *rol dentro de la estancia*. Un **peón** y un **admin** de la misma estancia tienen hoy los mismos permisos de lectura/escritura a nivel de fila — la columna **usuarios.rol** existe y está validada por CHECK, pero ninguna política RLS la usa todavía para diferenciar, por ejemplo, quién puede cerrar una alerta o modificar el estado de salud de un animal. Se decidió no implementarlo en el alcance de esta entrega (igual que la ausencia de auditoría centralizada, sección de decisiones de diseño del README) para priorizar el aislamiento multi-tenant, que es el riesgo de mayor impacto del caso de uso (fuga de datos *entre estancias*, no entre roles de una misma operación). La extensión natural sería una política adicional por operación sensible, por ejemplo: **CREATE POLICY alertas_resolver ON alertas FOR UPDATE USING (current_setting('app.rol') IN ('veterinario', 'encargado', 'admin'))**.

Otros controles: **contraseña_hash** nunca almacena contraseñas en texto plano (columna preparada para un hash bcrypt aplicado por la aplicación); las columnas de auditoría temporal (**fecha_creacion**, **created_at**) permiten reconstruir cuándo se creó cada registro aunque no haya una tabla de auditoría dedicada.

14. Consideraciones de escalabilidad y rendimiento

Desarrolladas en **arquitectura/escalabilidad.md**: qué tabla crece más (**mediciones**, por órdenes de magnitud sobre el resto), el particionamiento mensual ya implementado y su compromiso operativo (hay que crear particiones futuras con anticipación), la justificación de cada índice contra una consulta representativa concreta (no índices "por si acaso"), qué ya está precalculado (los dos campos desnormalizados) vs. qué se recalcula en cada consulta (las vistas, deliberadamente no materializadas al volumen actual), y qué se separaría si el volumen creciera un orden de magnitud (réplica de lectura, cola de ingesta, sharding por estancia).

15. Conclusiones

El diseño resultante cubre los cuatro tipos de datos del caso de uso (estructurados, semiestructurados, no estructurados de bajo volumen, vectoriales) en un único motor relacional, con cada decisión de modelado (normalización, desnormalización controlada, particionamiento, elección de índices) justificada contra una necesidad de consulta real y no en abstracto. Llevar el particionamiento, las políticas RLS y las vistas más allá del diseño en papel, hasta una implementación ejecutable, obligó a resolver tres puntos que el modelo lógico por sí solo no deja ver:

1. El particionamiento de **mediciones** necesita declararse explícitamente con **PARTITION BY** sobre la tabla real, no alcanza con documentarlo a nivel lógico.
2. El trigger de detección de anomalías necesita un mínimo de mediciones previas antes de evaluar, porque la desviación estándar de una muestra chica colapsa a 0 y generaría falsos positivos sistemáticos con poco historial.
3. Las políticas RLS solo tienen efecto si la aplicación se conecta con un rol sin privilegio de superusuario: **postgres** bypassa RLS por definición de PostgreSQL, por lo que la implementación agrega el rol **app_user** para que el aislamiento aplique de verdad.

Limitaciones reconocidas: permisos por rol dentro de una estancia (sección 13), rotación automática de particiones ([arquitectura/escalabilidad.md](#), sección 6), y ausencia de una tabla de auditoría centralizada (README, decisiones de diseño). Ninguna es necesaria para demostrar el diseño central del caso de uso — monitoreo IoT con detección de anomalías y aislamiento multi-tenant — y quedan planteadas como trabajo futuro.